Rudy: A Small Web Server

Dimitrios Chrysos

September 7, 2026

1 Introduction

This report evaluates the throughput of Rudy, a small web server under different conditions. Rudy was tested both locally and from a separate computer on the same local network.

A benchmark trial consists of sending a fixed number of sequential requests and measuring the time required to receive all responses. Throughput is calculated by dividing the number of requests by the average trial time.

2 Main problems and solutions

With few handlers and many concurrent clients, some connection attempts timed out because requests accumulated in the TCP backlog. Increasing the backlog reduced these timeouts by allowing more connections to wait.

3 Evaluation

3.1 Single-client Benchmarks

How many requests per second can we serve?

Each benchmark used five trials. Table 1 summarizes the measured throughput for local and remote clients. Trials on local clients consisted of 1000 requests while on remote clients 100 requests each.

Table 1: Single-client throughput.

Client	40 ms delay	Throughput (req./s)
Local	Enabled	24.37
Local	Disabled	6044.66
Remote	Enabled	19.20
Remote	Disabled	80.95

With the `timer:sleep(40)` delay enabled, the theoretical limit is

$$\frac{1 \text{ request}}{0.04 \text{ seconds}} = 25 \text{ requests per second.}$$

Is our artificial delay significant, or does it disappear in the parsing overhead?

The artificial delay dominates local processing overhead. Local throughput decreased from 6044.66 requests per second without the delay to 24.37 requests per second with it. The remote baseline was lower because each request created and closed a TCP connection over the local network. However, the 40 ms delay remained the dominant cost.

3.2 Simultaneous clients

What happens if you run the benchmarks on several machines simultaneously?

Two separate benchmark configurations were evaluated using four simultaneous clients. Two local clients on the server machine and two remote clients on the same local network. The first configuration enabled `timer:sleep(40)`, while the second configuration disabled it.

Each client performed five trials in both configurations. Trials with the delay enabled used 100 requests, whereas trials without the delay used 1,000 requests.

Table 2: Per-client throughput with four simultaneous clients.

Client	Delay enabled	Delay disabled
Local 1	6.27	4885.97
Local 2	6.20	5030.48
Remote 1	6.13	76.67
Remote 2	6.16	76.79

With the 40 ms delay, the combined throughput was 24.76 requests per second, close to the 25 requests-per-second limit with each client achieving approximately 6.2 requests per second. Without the artificial delay, the local clients achieved much higher throughput than the remote clients. This shows that TCP connection setup and local-network communication significantly affect remote clients when request handling is fast.

3.3 Handler pool

How much can you increase Rudy's request-serving throughput using multiple handler-processes compared to the single-threaded

implementation?

A fixed pool of Erlang handler processes allows delayed requests to be handled concurrently. The server was tested with pool sizes of 1, 2, 5, and 10 handlers. Ten concurrent clients performed five trials of 20 requests for each pool size, and `timer:sleep(40)` was enabled. The TCP backlog was increased to 100 so that more pending connections could wait safely.

Table 3: Handler-pool throughput with a 40 ms artificial delay.

Pool size	Theoretical (req./s)	Local (req./s)	Remote (req./s)
1	25	24.49	19.79
2	50	48.97	48.86
5	125	122.23	120.90
10	250	242.49	129.85

Local throughput scales almost linearly with the handler-pool size and remains close to the theoretical maximum in every experiment. Remote throughput improves up to five handlers, reaching 120.90 requests per second. Increasing the pool from five to ten handlers increases remote throughput only slightly, to 129.85 requests per second. Therefore, five handlers are more appropriate for this remote workload, since additional handlers provide little benefit while still requiring additional system resources. This also shows that 10 remote clients cannot generate requests quickly enough to fully utilize the larger handler pool of 10 handlers

Multiple handler-processes could have also been achieved by spawning a new process for each accepted client connection. However, creating a process for every connection adds a small overhead and can increase memory usage when many clients connect simultaneously. For this reason, a fixed-size handler pool was preferred.

4 Conclusion

Rudy demonstrates that concurrent Erlang handler processes can substantially increase throughput when request handling includes delays. Network and TCP connection overhead become a main limitation for remote clients when the server has many available handlers.